

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Классы и ООП

ООП – это объектно-ориентированное программирование, представляет собой способ программирования, который базируется на использовании объектов и классов для проектирования и создания приложений.

В Python всё является объектом – числа, списки, кортежи, строки, классы и т.д. Чтобы создать объекты, используются классы.

Класс - это схема, описывающая структуру, данные внутри неё, и присущие ей методы. То есть это шаблон для создания объектов.

Метод - это функция, то есть метод объекта - это функция, описанная внутри объекта, и присущая этому объекту, которая действует на объекты данного вида.

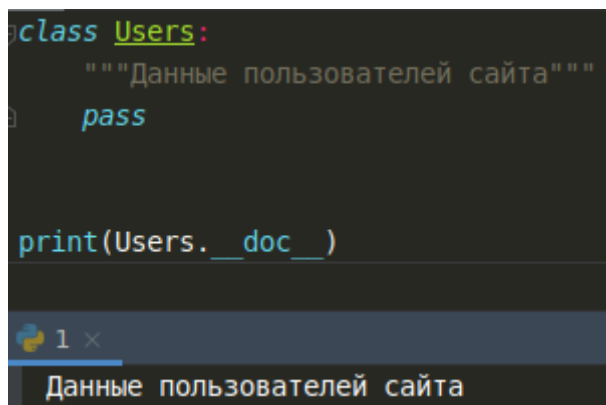
Экземпляр - это конкретный объект, созданный из класса.

Создание класса

Синтаксис для написания нового класса:

Определения класса в языке Python начинается с зарезервированного слова **class**, и следующего за ним именем класса, которое должно быть с большой буквы. Под именем класса может быть (необязательно) краткое описание класса в тройных двойных кавычках (строка документации). Получить доступ к этой строке можно с помощью **ClassName.__doc__**.

Пример пустого класса.



```
class Users:
    """Данные пользователей сайта"""
    pass

print(Users.__doc__)
```

1 x

Данные пользователей сайта

В теле класса допускается объявление атрибутов, методов и конструктора.

Атрибут (поле класса) — это элемент класса, например, у пользователей это могут быть такие данные как e-mail, login, password. То есть атрибутами в классах выступают переменные внутри класса или, например, параметры, передаваемые в функцию.

Также существуют и встроенные атрибуты класса, например, **__doc__**, который мы использовали выше. Этот атрибут возвращает строку с описанием класса, или None, если значение не определено.

Доступ к пользовательским атрибутам класса мы можем получить аналогично встроенным атрибутам – вызвать класс, где через оператор точка будет указан

нужный атрибут. В данном примере password будет являться атрибутом класса (объекта) Users. А Users.password является ссылкой на атрибут.

```
class Users:
    """Данные пользователей сайта"""
    login = 'admin'
    password = '12345'
    email = 'superadmin@gmail.com'

print(Users.password)
```

1 x

12345

Создадим объект класса Users:

`admin = Users()`

Объект — это экземпляр класса. В данном случае admin является объектом класса Users. Таким образом мы получили пустой объект класса, через который мы можем получать доступ ко всем атрибутам класса через точку.

```
class Users:
    """Данные пользователей сайта"""
    # атрибуты класса
    login = 'admin'
    password = '12345'
    email = 'superadmin@gmail.com'

# создаем экземпляр класса Users (объект)
admin = Users()
print(admin.email)
print(admin)
```

1 x

superadmin@gmail.com

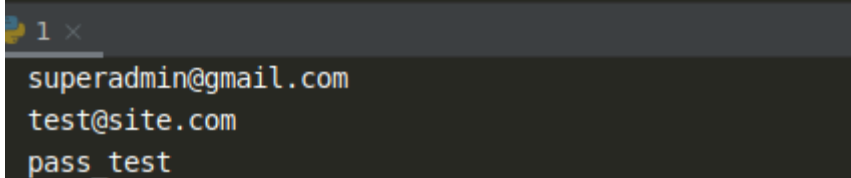
<__main__.Users object at 0x7feb391ea048>

Мы можем создать сколько угодно экземпляров класса, при этом присвоив им совершенно произвольные значения (атрибуты экземпляра).

Атрибуты класса **одинаковы для всех экземпляров** класса, тогда как **атрибуты экземпляров** являются уникальными для каждого экземпляра.

```
class Users:
    """Данные пользователей сайта"""
    # атрибуты класса
    login = 'admin'
    password = '12345'
    email = 'superadmin@gmail.com'

admin = Users() # создаем экземпляр класса Users
testuser = Users() # создаем экземпляр класса Users
testuser.email = 'test@site.com'
testuser.password = 'pass_test'
print(admin.email)
print(testuser.email)
print(testuser.password)
```



Помимо полей (атрибутов), пользовательский класс может включать в себя и методы, которыми будут наделены все его экземпляры. Вызвать выполнение определенного метода через созданный объект можно так же, как и получить доступ к его полям, то есть с помощью точки.

Методы класса

Методом называется функция, определённая внутри класса, в которой первым ОБЯЗАТЕЛЬНЫМ параметром идёт ключевое слово **self**. Имя **self** - это соглашение, **self** нужен для того, чтобы обращаться к любым атрибутам класса.

На картинке ниже, мы создали функцию **user**, в которую передали несколько параметров. А с помощью **self**, через точку обратились к атрибутам класса, и приравнивали наши параметры к атрибутам класса.

```
def user(self, login, password, email):
    self.login = login
    self.password = password
    self.email = email
```

Далее имена пользователей через точку связываются с функцией (используем метод **user**), а в скобках мы указываем нужные нам аргументы.

```
admin.user('admin', '12345', 'superadmin@gmail.com')
```

И теперь мы можем вызывать нужный объект с любыми описанными параметрами.


```

class Users:
    """Данные пользователей сайта"""
    # атрибуты класса
    login = ''
    password = ''
    email = ''

    def user(self, login, password, email):
        self.login = login
        self.password = password
        self.email = email

admin = Users()
admin.user('admin', '12345', 'superadmin@gmail.com')
testuser = Users()
testuser.user('testuser', 'pass_test', 'test@site.com')
print(admin.login, admin.password, admin.email)
print(testuser.email)

```

1 x

```

admin 12345 superadmin@gmail.com
test@site.com

```

Методов можно создать неограниченное количество. Добавим ещё один метод, который будет выводить все атрибуты экземпляра в отформатированном виде.

```

class Users:
    login = ''
    password = ''
    email = ''

    def user(self, login, password, email):
        self.login = login
        self.password = password
        self.email = email

    def all_user(self):
        print(f"User: {self.login}\nPassword: {self.password}\nEmail: {self.email}\n")

admin = Users()
admin.user('admin', '12345', 'superadmin@gmail.com')
testuser = Users()
testuser.user('testuser', 'pass_test', 'test@site.com')
admin.all_user()

```

Вывод программы:

```
User: admin
Password: 12345
Email: superadmin@gmail.com
```

Конструктор класса

Чтобы заранее определить атрибуты для объекта, задав их во время его создания, применяется конструктор. Он автоматически срабатывает каждый раз, когда программа создает новый объект для класса, в котором он расположен. Конструктор – это специальный метод класса, называется `__init__`.

Первый параметр конструктора во всех случаях `self`. При использовании метода `__init__`, в сравнении с пользовательским методом `user`, код сокращается, так как не требуется писать атрибуты класса в теле.

```
class Users:

    def __init__(self, login, password, email):
        self.login = login
        self.password = password
        self.email = email

admin = Users('admin', '12345', 'superadmin@gmail.com')
testuser = Users('testuser', 'pass_test', 'test@site.com')
print(admin.login, admin.password, admin.email)
print(testuser.login, testuser.password, testuser.email)
```

test x

```
admin 12345 superadmin@gmail.com
testuser pass_test test@site.com
```

Атрибуты экземпляра можно вывести с помощью встроенного метода `__dict__`.

```
print(admin.__dict__)
```

test x

```
{'login': 'admin', 'password': '12345', 'email': 'superadmin@gmail.com'}
```

Инкапсуляция

По умолчанию все свойства классов открыты для доступа извне, благодаря чему их можно в любой момент изменить по своему усмотрению при помощи оператора точки.

```
class Users:
    login = 'admin'
    password = '12345'
    email = 'superadnin@gmail.com'

admin = Users()
admin.password = '123'
print(admin.password)
```

1 x

123

Инкапсуляция предотвращает прямой доступ к атрибутам объекта, то есть изменение данных вне класса. Инкапсуляция в Python работает лишь на уровне соглашения между программистами о том, какие атрибуты являются общедоступными, а какие внутренними (защищёнными или приватными).

Одинарное подчеркивание в начале имени атрибута говорит о том, что переменная или метод не предназначен для использования вне методов класса, однако атрибут доступен по этому имени.

```
class Users:
    _login = 'admin'
    _password = '12345'
    email = 'superadnin@gmail.com'

admin = Users()
print(admin._password)
```

1 x

12345

Чтобы ограничить видимость полей (атрибутов), следует задать для них имя, начинающееся с двойного подчеркивания.

```
class Users:
    __login = 'admin'
    __password = '12345'
    email = 'superadnin@gmail.com'

admin = Users()
print(admin.__password)
```

При попытке обращения к приватному атрибуту, мы получим ошибку.

```
Traceback (most recent call last):  
  File "/root/PycharmProjects/education/1.py", line 8, in <module>  
    print(admin.__password)  
AttributeError: 'Users' object has no attribute '__password'
```

Казалось бы, обращение напрямую теперь закрыто, однако это соглашение о приватности можно обойти.

```
admin = Users()  
print(admin._Users__password)  
1 Hack!  
12345
```

Таким образом, мы видим, что реального сокрытия объектов нету, всё на уровне соглашений. Подчёркивания сообщают другим программистам, что данные атрибуты предназначены для служебного пользования, и их изменение может привести к нарушению работоспособности программы.